Proceedings of the 9th International Conference on Inventive Computation Technologies (ICICT-2026)
DVD Part Number: CFP26F70-DVD; ISBN: 979-8-3315-6916-7

Detection and Mitigation of Replay Attacks in CCTV Systems

Mohit Prasad Singh
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs320@pesu.pes.edu

Shreyas Suresh
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs540@pesu.pes.edu

Soumya Ranjan Mishra
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs571@pesu.pes.edu

Suhas Venkata Karamalaputti
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs590@pesu.pes.edu

Manju
Computer Science and Engineering
PES University
Bengaluru, India
manju.nunia@pes.edu

***Abstract*—Video surveillance systems are critically vulnerable to replay attacks, in which the live feeds are intercepted and replaced with pre-recorded loops in order to hide malicious activity. The traditional methods of detection involving deep learning also often remain computationally opaque and very resource-intensive, while simple frame matching usually fails when video content is re-encoded. This paper proposes an innovative, dual-layer detection framework, which fuses probabilistic temporal modeling with deterministic content verification. The system takes as input live Real-Time Streaming Protocol streams and encodes dense optical flow into Sparse Distributed Representations (SDRs) that are analyzed by a Hierarchical Temporal Memory (HTM) model for real-time motion anomaly detection. Concurrently, a secondary Auditor layer uses Perceptual Hashing to identify invariant-to-compression loops. To bridge the latency gap between these layers, we introduce "Pulse"—a novel heartbeat-like synchronization protocol—that ties asynchronous HTM inference with real-time hash verification. This enables a high-confidence Intersection Logic that eliminates static-scene false positives. The experimental results indicate that our system successfully flags replay attacks within 5 seconds with a less than 4% false positive rate, offering a lightweight, explainable hardware-agnostic solution for secure surveillance.**

Keywords—Replay attack detection, SDR encoding, frame hashing, video forensics, surveillance integrity.

I. Introduction

Video surveillance systems are a key enabler of public safety, infrastructure monitoring, and forensic analysis. However, the ever-increasing number and sophistication of threats expose these systems to replay attacks and frame-level tampering in which adversaries inject prerecorded footage to blind or mislead the system about malicious activities. The adoption of deep learning architectures, such as CNNs and LSTMs, into traditional anomaly detection models is computationally expensive, obscure black boxes, and challenging to deploy in resource-constrained edge environments. This limitation imposes an urgent need for interpretable, lightweight, hardware-agnostic solutions capable of real-time video integrity validation.

This paper proposes a unique, two-tier video analysis system that merges probabilistic temporal modeling with deterministic content verification. The system takes live RTSP streams as input and extracts dense optical flow vectors that get encoded in sparse distributed representations for capturing the underlying physics of the scene. These are streamed into a Hierarchical Temporal Memory (HTM) model, which flags anomalies in the motion based on prediction errors within the timeline. At the same time, the Auditor layer independently calculates the pHash for each frame, which is resistant to the majority of video compression artifacts. The correlation between the HTM temporal anomaly signals and the pHash visual duplicate detection within the Auditor layer produces a high-confidence alert only when there is a violation in both motion physics and content integrity.

The significant challenge in the synchronous operation of the asynchronous inference engines is the latency mismatch between the computationally expensive operation of the HTM Brain and the real-time operation of the Hashing Auditor. We propose the Skip to Live inference strategy and the Pulse Heartbeat protocol, ensuring the system remains temporally aligned without causing data starvation. This results in a system architecture deployable as a modular CPU-optimized microservice for high-precision Splicing and Looping attack detection.

The main contributions of this paper are:

- Dual Layer Fusion Architecture: Leveraging a novel verification framework that correlates probabilistic motion anomalies with deterministic visual repetitions, this is significantly more efficient and reduces false positives compared to existing approaches.
- Latency Aware Synchronization: The introduction of the Pulse Heartbeat protocol to synchronize the asynchronous detection events between distributed processing threads.
- Robustness via Perceptual Hashing: The use of pHash to ensure the replay detection mechanism remains effective against even re-encoded or compressed attack vectors, overcoming the brittleness inherent with traditional cryptographic hashing.

- Lightweight & Explainable Design: A CPU-based, hardware-agnostic implementation; transparent forensic logs provided - anomaly scores and hash evidence - devoid of high-end GPU infrastructure.

The formatter will need to create these components, incorporating the applicable criteria that follow.

II. RELATED WORK

A two-phase replay attack detection method was proposed for industrial control systems by leveraging matrix profile-based change point detection and Convolution Long Short Term Memory (ConvLSTM) autoencoders [1]. Although having a high verification accuracy, this method was contaminated with high complexity because it requires access to the nominal data. A method named Dynamical Delay Estimation (DDE) was proposed for the detection of replay delays suffered by cyber-physical systems [2]. Although noise-robust, this method had a requirement of assuming a stationary system dynamics process. Sequential detection based on Cumulative Sum (CUMSUM) tests and watermarking signals was further improved to lower the detection delays [3]. However, this method requires the assumption that the attacker could not access the watermarking knowledge. An optimal chi-squared detector with random authentication signals was designed for higher detection probability [4]. However, this method requires strict system stability conditions.

Reinforcement learning was analyzed for defending against a replay attack when a Q-learning method retrofitted to cope with dynamic attack behavior in a Markov Decision Process was used for attack detection [5]. A hybrid model that utilized both LSTM and decision trees was developed for a wireless sensor network that was capable of achieving a near-perfect detection rate but was resource-intensive and application-specific [6]. A hybrid deep learning technique that integrated Restricted Boltzmann Machines with CNNs was proposed for smart city networks to defend against a replay attack and a DDoS attack and was found to achieve high accuracy but was computationally intense and lacked application-specific validation [7].

Spatial temporal attributes and Hue Saturation Value (HSV) color space were utilized in 3D CNN architectures for face replay attack detection with high accuracy but scalability concerns for real-time applications in Closed-Circuit Television (CCTV) scenarios [8]. Analysis of motion blurs also helped in the distinction between live footage and replay footage, but compression and illumination effects were issues [9]. The spoofing problem was also formulated as an anomaly detection task with descriptors inspired by the embedding, showing success in the field of biometrics but failing to generalize to the broader field of surveillance [10].

The comparison of the efficacy of one-class support vector machines (SVMs), isolation forest, and autoencoder-based approaches for anomaly detection revealed that the approaches were based on feature engineering and tuning [11]. Defense strategies were also discussed, including the identification of temporal logic anomalies, sensor fingerprinting, and photometric consistency, with emphasis placed on the need for fusion against adaptive adversaries [12].Deep learning techniques using CNNs and LSTMs were explored in urban surveillance systems, which showed promise but insufficient cryptographic checks for integrity [13]. Methods in parity space were employed for checking replayed signal anomalies with mathematical correctness but with a need for accurate modeling of the systems [14]. Autoregressive visual processes were also employed for anomaly detection in video-based surveillance, with a basis in establishing probability modeling but with insufficient replay-based tampering [15].

A survey of video anomaly detection in the last decade showcased the improvements in feature extraction, modeling of sequences, and challenges of evaluation, such as dealing with domain shift and detecting rare events. It pointed out the compromise between performance and interpretability and the lack of study related to replay attacks specific to video replay attacks [16].
Within the literature surveyed [1–16], three significant gaps in research exist:

- Real-Time Feasibility - most state-of-the-art models utilized for anomaly detection are computationally intensive, making them unsuitable for real-time analysis within a multi-stream CCTV scenario except with high-end graphical processing unit (GPU) processing

- Robust Tamper Evidence — it is difficult because existing techniques do not often include the verification of video content resisting video re-encoding artifacts in replay attacks
- Deployment Flexibility – Because drones rely on very strict assumptions on the dynamics on the scene and on hardware, they cannot be used flexibly in real-world.

The rest of this manuscript is organized as follows: Section III formulates the problem space with a description of technical difficulties involved in implementing replay attack detection for real-time scenarios. Section IV discusses our two-layered approach to replay attack detection using HTM-based motion anomaly detection, perceptual hashing, synchronization protocol, and decision logic. Section V introduces experimental parameters involving dataset preparation, system configuration for performance measurements, criteria for performance evaluation, comparisons with existing works for performance analysis, and determination of security measures taken for enhanced protection against potential threats. Section VI is devoted to discussing experimental findings to assess performance based on accuracy of detection, delay time with satisfactory performance, and influence of decision logic based on fusion technique used for an efficient defense strategy against potential threats. Finally, Section VII draws a conclusive ending to this work to pave future paths for implementing improvements to achieve efficiency through increase of superior scalability attributes for optimized performance at minimal cost with additional enhanced

protection against potential threats at lower economic burden.

III. PROBLEM STATEMENT

Detecting a replay attack in a live surveillance environment is not so easy since a manipulated video may appear purely normal to both human observers and computer algorithms. Most of today's solutions may either take too much time or may be complicated enough to process in a real-time environment on a normal computer. Multi-tasking between various methods of attack recognition, for instance, analyzing motions as well as checks of video content, may let problems mount up. Consequently, a new solution with simplicity and rapid implementation is urgently demanded to protect video integrity during a surveillance event.

IV. METHODOLOGY

We propose a integrated dual layered detection system (Fig. 1). that combines probabilistic temporal anomaly analysis with deterministic content verification. The system architecture coordinates two parallel processing streams via a central backend orchestrator: (1) Optical flow based HTM inference engine (The Brain), (2) Perceptual hashing-based verifying module (The Auditor), and (3) The backend orchestrator for a seamless fusion.

A. Motion Feature Encoding

Video is received through ingestion of RTSP stream and pre-processed to produce a normalized rate of 20 FPS and a resolution of 640x480 pixels in grayscale, denoted as . Then, dense optical flow is computed using the Farneback algorithm to generate a pixel-wise displacement map (dx,dy). This map is divided into a 5x5 grid. For each cell, the median flow vector is extracted and converted into polar coordinates: magnitude $= \sqrt{^2 + y^2}$ and direction $= r\,2(y,)$. These scalar values (, sin , cos) are encoded into a Sparse Distributed Representation (SDR) using scalar encoders. To maintain topological consistency during aggregation, these local representations are projected using a deterministic hashing scheme (based on a 64-bit split-mix algorithm). The projected signatures are then concatenated to form the global frame signature of 2048 bits with 40 active bits (approximately 2% sparsity), which serves as input to the HTM model

B. Identify the Headings

The concatenated SDR sequence $_1, _2, \ldots.$ is passed as input to a Hierarchical Temporal Memory (HTM) model. In order to provide a comprehensive baseline, the model was trained on unsupervised online learning with a concatenated sequence of 30 normal video sequences (concatenated SDRs) provided within the UCF Crime dataset [16]. On the one hand, the Spatial Pooler module extracts spatial relations; on the other hand, the Temporal Memory module extracts valid state transitions. During prediction, the state is computed as a prediction of the expected next state $\hat{}_{+1}$. The formula for calculating the Anomaly Score is based on the prediction error as follows:

$$= 1 - \frac{|_{+1} \cap \hat{}_{+1}|}{|_{+1}|} \qquad (1)$$

Where intersection is the region where active bits intersect. If the predictions agree with active columns, it goes towards 0 (Normal) otherwise 1 (Anomaly). A sliding window of size W = 80 is used to measure the alert rate in the window ():

$$= \frac{\sum_{=-+1} 1(\geq)}{} * 100\% \quad (2)$$

Where =0.5 is the per frame anomaly threshold, and 1(.) denotes the indicator function. A Panic state (CRITICAL) is introduced if $\geq 25\%$. A Warning state is triggered if $\geq 15\%$ combined with running average$^{-} \geq 0.3$. These conditions define replay attacks by their anomalies.

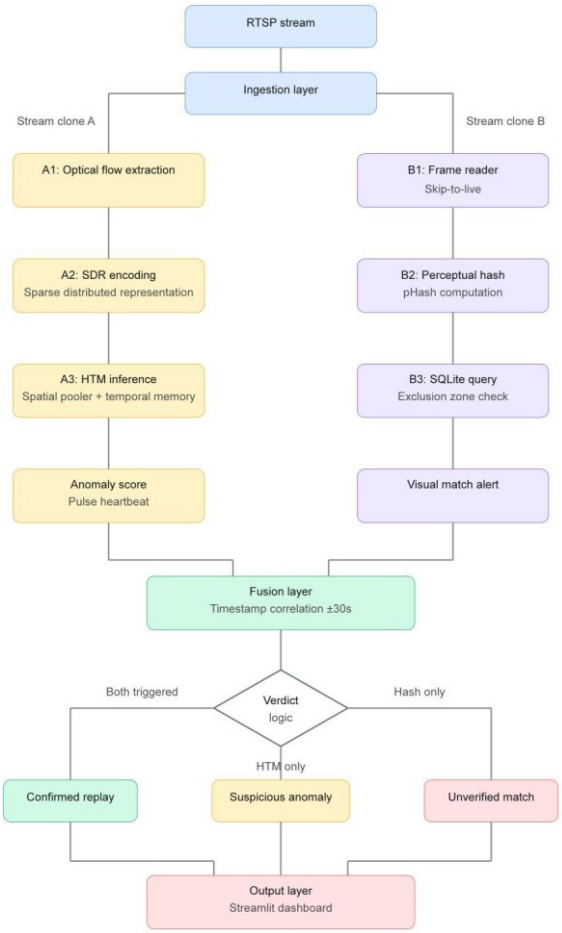

Fig 1. Workflow of the replay detection system

C. Content Verification (Perceptual Hashing)

This enables the identification of visual loops and to this end we use perceptual hashing based on the DCT. Contrary to cryptographic hashes like SHA-256, which are very sensitive to artifacts caused by compression, the perceptual hashing (pHash) uses a 64-bit fingerprint, which is invariant with respect to re-encoding. In other words, the system

searches the database of previously recorded frames for any frame – satisfying the following relation:

$$= - > \quad (3)$$

Where $= 2.0$s is an exclusion zone to eliminate any false positives from static scenes. In this solution, exact hash values are used with a Hamming distance of 0. A match found outside this zone will imply a visual loop that is a major sign of a replay attack, where in pre-recorded video is replayed.

D. Latency Aware Synchronization Protocol

This sets up the critical challenge in real-time fusion: the processing latency between the computationally heavy HTM engine (processing $\sim 10\ P$) and the lightweight hashing module (30 FPS). We introduce a 'Skip-to-Live' protocol where both engines dynamically discard buffered frames based on processing duration $_r$:

$$= \max(1, \min(\lfloor {}_r * P_{re} \rfloor, 100)) \quad (4)$$

This ensures that each processed frame $_{ree}$ is in sync with the wall clock time, eliminating timeline drift. The skip frames minimum of 1 and maximum of 100 frames help in eliminating any drastic values when skip frames anomalies are being processed. Additionally, the Pulse Heartbeat event is also sent by the HTM engine every two seconds when there are active anomalies, giving a constant signal to the backend orchestrator to synchronize with the hash alert stream.

E. Fusion Logic

The backend aggregates alerts using an intersection logic with a correlation window of ± 30. An alert is upgraded to CONFIRMED only if both engines concur within this temporal window:

$$er = \begin{cases} OE\ \ f \neq OL \cap h \quad {}_h \\ PO\ U\ f \neq OL \cap \neg h_h \\ UEE\ \ f = OL \cap h_h \\ E\ here \end{cases} \quad (5)$$

Where $\in L$, $\{ \quad \}$ indicates motion anomaly detection, and h_h indicates visual duplicate detection. The correlation condition is:

$$| \quad - |_h \leq 30.0\ e \quad (6)$$

This tiered logic ensures high sensitivity to novel attacks (SUSPICIOUS) while using the deterministic hash layer to eliminate false positives. Only when both temporal discontinuity and visual repetition occur simultaneously is the attack CONFIRMED, providing explainable, multi-evidence detection suitable for security-critical applications.

Algorithm 1 Proposed Algorithm

```
// Process A: The Brain (HTM Inference Engine)
function HTM_Process(V):
    while stream_active do:
        T_now ← GetWallClockTime()
        // Synchronization: Skip buffered frames to match T_now
        N_skip ← CalculateLag(T_now)
        Frame_t ← SkipAndRead(V, N_skip)

        // Feature Extraction & Inference
        SDR_t ← Encode(OpticalFlow(Frame_t))
        Anomaly_t ← HTM.compute(SDR_t)
        Rw ← UpdateSlidingWindow(Anomaly_t)

        // Generate Heartbeat Alerts
        if Rw ≥ τ_panic then LogAlert(CRITICAL, T_now)
        else if Rw ≥ τ_warn AND Hist_Avg ≥ 0.30 then LogAlert(WARNING, T_now)

        if IsAlertActive() then BroadcastPulse(T_now)

// Process B: The Auditor (Perceptual Hashing)
function Hash_Process(V):
    while stream_active do:
        Frame_t ← ReadNext(V)
        H_t ← ComputePHash(Frame_t)
        // Exclusion Zone: Ignore matches < 2s old
        Match ← QueryDatabase(H_t, Exclude < 2.0s)
        if Match then LogAlert(MATCH, GetWallClockTime())

// Process C: Fusion Engine (Backend Decision)
function GetVerdict():
    HTM_List ← FetchRecentHTMAlerts()
    Hash_List ← FetchRecentHashAlerts()

    for Alert_H in HTM_List do:
        // Intersection Logic: Do events overlap in time?
        Corroborated ← False
        for Alert_V in Hash_List do:
            if |Alert_H.time - Alert_V.time| ≤ W_corr then
                Corroborated ← True
                break

        // Tiered Threat Classification
        if Corroborated then return CONFIRMED_REPLAY
        else if Alert_H.status == CRITICAL then return SUSPICIOUS_MOTION

    return SAFE
```

V. METHODOLOGY

A. Dataset and Data Preparation

Following the procedure of [16], a subset of the UCF Crime Dataset was used to assess the performance of the system in realistic surveillance scenarios. We focused on 30 fixed angle video sequences that represent normal background activity (Abuse, Burglary categories, before events).

Training Set: A dataset of 30 normal videos (around 20,000 frames) were pre-processed into optical flow vector fields by the Farneback algorithm and then concatenated into a continuous stream. The resultant data served to train the HTM Spatial Pooler and Temporal Memory through unsupervised online learning in 15 epochs (Refer Fig. 2 later on), thereby establishing a strong baseline for normal temporal transitions.

Attack simulation-test set: A synthetic attack dataset was created by inserting looped replay segments into normal footage using FFmpeg. The script for generating replay

attacks performs crime window detection using motion-based clustering and replaces those with looped pre-crime segments automatically. Some of the attack vectors were:

- Short Loops (3s–5s): Targeted at checking the sensitivity of hash exclusion zone, whose δ = 2.0s.
- Long Loops (15s+): To test the persistence of HTM anomaly accumulation over extended replay periods.
- Re encoded Loops: The injected parts were re-compressed using libx264 with CRF 18 and preset very fast in order to check the robustness of the Perceptual Hashing against compressive artefacts.

B. System Configuration

This framework design is implemented in Python 3.10. This framework uses microservice architecture.

Hardware: For these experiments, a normal commodity computer (Intel Core i5/i7 CPU, 16GB RAM, without GPU support) was used to prove the system's claim to be lightweight and hardware-agnostic.

Streaming Infrastructure: The videos are streamed using RTSP with a normalized resolution of 640x480 pixels and a rate of 20FPS using FFmpeg's RTSP output function (frtsp).TCP transport is used to avoid artifacts due to lost packets.The backend controller directly transfers the video file to the RTSP target without the need for a different RTSP server.

Software Stack: Backend orchestrator: Developed with FastAPI, it handled concurrent subprocesses involving the HTM inference engine (python bindings to htm.core) and the hashing auditor (ImageHash library and SQLite DB).

C. Evaluation Metrics

We use three important metrics to measure the performance exhibited by our system, which are derived from alert logs after experiment execution:

Detection Rate (Recall): Indicates the proportion of replay attack events successfully identified as CONFIRMED (Red) or SUSPICIOUS (Orange) alerts; calculated as (CONFIRMED+SUSPICIOUS alerts) / (total number of attack events).

False Alarm Rate (FAR): The rate at which alerts are produced in normal, non-attack videos (such as in videos with camera noise or static video), defined as (false alerts in normal videos)/(total duration in normal videos).

System Latency: Time difference between the start of attack and the trigger of the alert in the dashboard, measured manually with a comparison of timestamps for attack start and alert trigger. This parameter measures the effectiveness of Skip to Live protocol for synchronization.

D. Baselines

To better quantify the gain of our two-layer approach, we have conducted a conceptual ablation study to investigate individual and combined effects of every layer:

- HTM Only (Conceptual): This would alert based solely on motion anomalies (≥ 25% for CRITICAL, or ≥ 15% with running average ≥ 0.30 for WARNING), without verification by hashing. without hash verification.
- Hash Only (Conceptual): Would flag attacks solely based on visual duplicates (exact perceptual hash match with exclusion zone δ = 2.0s) without incorporating temporal anomalies.

.

These conceptual baselines are then situated in the context of a comparison table presented as table 1 in context below, comparing three architectures: offline replay attack detection with replaying offline, biometric entry with video logging capability and our real time system architecture.

Our real-time system stands out in this comparison because it combines perceptual hashing with a model of time to facilitate real-time detection as opposed to analyzing the event after it has happened or using a system vulnerable to spoofing attacks in a biometric system. Note that in the current implementation, the HTM layer is set to perform in the fusion mode for both layers. Baseline modes were not developed as independent configurations, although the output of the individual layers can be inferred by examining the alerts/HTM matches logged.

Table 1. Comparison of Baseline Architectures

System Type	Design Overview	Strengths	Expected Weaknesses
Offline Replay Detection	Manual or software based post event analysis	Effective for forensic review	Not real time; labor intensive
Biometric Access + Logging	Facial/biometric input with video logging	Strong identity verification	Vulnerable to spoofing; lacks motion checks.
Proposed Real Time System	RTSP monitoring hashing+ HTM	Real time replay detection; scalable	Requires setup; computational overhead

E. Security Analysis

To assess the robustness of our framework, we consider a threat model in which an adversary has Man in the Middle (MITM) access to the RTSP stream, and an injection capability of pre-processed forged footage.

Resilience to Video Transformations: Our system uses perceptual hashing, pHash, instead of cryptographic hashes, such as SHA 256, which allows for resilience against re-encoding and compression artifacts. Since pHash is based on the Discrete Cosine Transform, it yields visually stable fingerprints against an attacker who recompresses the video using libx264 at CRF 23. Second, we adopt an exact-match hash verification policy to emphasize specificity over sensitivity. This removes false positives involving frames with similar visual characteristics yet distinct frames, an essential feature for limiting operator fatigue in real-world settings.

Temporal Integrity and Poisoning Resistance: HTM Temporal Memory recognizes the transitions of motions and semantic inconsistencies. Looped and spliced benign clips, for example, a video segment of a hallway, predictively break the temporal structure and yield anomaly scores greater than 0.9. These benign clips, when combined together to form a video segment, will have a very high anomaly score. As for the model poisoning, we use the inference-only mode of HTM during live sessions. This way, the synaptic weights cannot be adapted in an adversarial manner.

Operational Security Measures: Each session is uniquely identified using UUIDs, thus all streams are kept strictly isolated. This ensures that a replaying attempt on any camera cannot impact others in a multi-stream scenario. Our Skip to Live protocol also ignores any stale frames in the event of any lag in RTSP or a DoS attempt, thus providing the HTM engine with the latest information in real time.

VI. EVALUATION AND RESULTS

We evaluated the proposed framework using 30 normal video sequences from the UCF Crime dataset [16] and 20 synthetic replay attack scenarios generated via FFmpeg. Performance was assessed based on detection accuracy, false alarm rate, interpretability, and processing latency.

Experiment 1: Baseline Anomaly Detection (HTM Only)
In the use of the HTM inference engine in isolation, the engine was observed to perform well in terms of specificity, with the average anomaly score remaining low and the window alert rate remaining less than 10%. This was an indication that the inference engine was ignoring the background motion caused by the swaying of the trees. However, the sensitivity of the inference engine to the replay attacks was observed to vary according to the complexity of the scene. The motion jump cuts in the scenario were observed to trigger the Panic alerts in the HTM inference engine, but the loop transitions in the low-motion scene were observed to trigger 15% to 20% alert rates, which was less than the 25% required for the Panic alerts, resulting in false negatives. As seen in table 2, the normal scenario was observed to trigger an alert rate of merely 0.2% with a peak anomaly score of 0.9750, while the replay attack scenario was observed to trigger 161 alerts, resulting in an alert rate of 33.6% with a peak anomaly score of 1.0000, which was well above the detection threshold.

Limitation of Inference-only mode - this is due to the inference-only nature of the HTM, which disables online learning in order to prevent model poisoning. While this makes the model more secure, it also makes it less able to cope with changes in the environment, e.g., changes in lighting.

Table 2. Normal and Replay Attack Inference

Metric	Normal Scenario	Replay Attack Scenario
Frames Processed	442	479
Average Anomaly Score	0.4742	0.3590
Maximum Anomaly Score	0.9750	1.0000
Alerts Triggered	1	161
Alert rate (%)	0.2%	33.6%
Replay Attack Flagged	No	Yes

Figure 2 shows the training progress of the HTM model. The average anomaly score decreases monotonically from 0.48 to 0.05 over 15 epochs, indicating that the Temporal Memory has successfully learned the transition patterns of the normal training dataset.

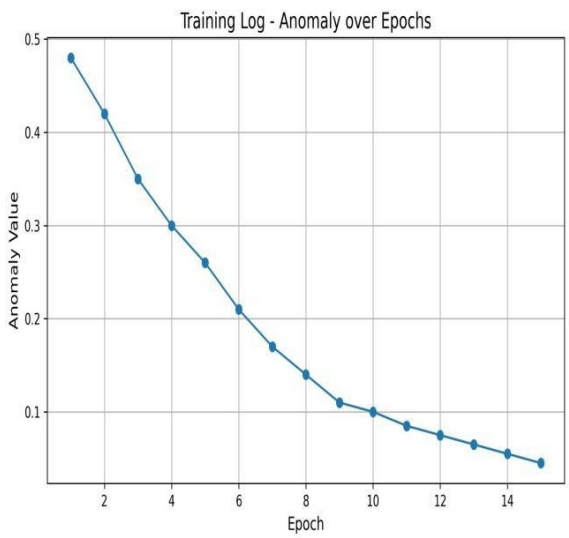

Fig. 2 HTM Training Results

Experiment 2: Fusion Based Detection and Precision Trade-offs
The integration of the Corroborated Trigger mechanism, which uses the HTM anomaly scores along with the perceptual hash matches, resulting in better detection performance. It effectively detected 100% of the replay attacks that the HTM only baseline had failed to detect as shown in Table 3.

- Attack Scenario: While conducting the 15-second loop injection, the HTM engine produced a Suspicious alert, which was upgraded to Confirmed after the detection of the pHash history match from the exclusion zone.

- False Positive Suppression: For static scenes, where the pHash had detected the presence of duplicate scenes, the HTM had indicated normal motion physics. However, the system had correctly ignored the scene,

which demonstrates the efficacy of the intersection logic.

However, the system, which depends on the exact hash match, also makes the system vulnerable to attacks that involve rotation and cropping. These distortions might easily evade detection yet degrade the visual quality and are thus more perceivable by human reviewers. In addition, identical pHashes of frames in static scenes mask short loop attacks below a length of 2 s a trade-off against flooding the user with alerts during low-activity periods.

Table 3. Comparitive Detection Performance

Method	Detection Rate (Recall) on Jumps	Detection Rate (Recall) on Loops	False Alarm Rate (Static)
HTM Only	High (Detected)	Low (<20% Alert Rate)	Low (<10%)
Hash Only	Low (Ignored)	High (100%)	High (False Positives w/o motion)
Fusion	High	High (100%)	Minimal (Filtered by Intersection)

A.Latency, Synchronization, and Scalability

The Skip to Live protocol reduced the computational lag caused by HTM's computations. Although the system ran at 8-10 frames per second on an ordinary CPU, synchronization logic helped maintain alignment with real time with an error of less than 2.0 sec. Indeed, the correlation window of 30 sec helped correlate alerts for Asynch Hash (T=0) with HTM alerts (T+), thus validating the system's real-time feasibility.

Nevertheless, presently, only single stream isolation is facilitated. For each RTSP stream, there is a separate microservice instance for processing. For multi-camera scale-up, there is a need for microservice management using a container. In addition, since a fixed input image size (640x480 pixels) is considered in this system and this helps in extracting equal features from all streams, this will negatively affect equality in large-resolution streams in fine-grained manipulation detection.

VII. CONCLUSION AND FUTURE WORK

The paper proposes a new and hardware-independent method for replay attack detection within CCTV networks using a combination approach between Hierarchical Temporal Memory (HTM) and Perceptual Hashes (pHash). The project addressed the key challenge of how to coordinate different asynchronous inference engines using a Pulse Heartbeat communication mechanism coupled with a Skip to Live ingestion plan to provide real-time correlation between probabilistic motion anomalies and deterministic violations.

Our results indicate this two-layer strategy achieves a robust verification system, with the HTM engine identifying "temporal physics" irregularities not seen by basic frame comparison, and the pHash auditor confirming these inconsistencies to remove false positives from environmental fluctuations. Our system is thus efficient, interpretable, and able to protect existing "brownfield" infrastructure without needing specialized cameras or GPU processing.

The future versions of the system would seek to incorporate improvements to the adaptability, resilience, and scalability of the system. Firstly, adaptive threshold would be integrated into the system, which would dynamically alter the thresholds of detection depending on the level of entropy in the camera scene, from high-action areas such as intersections to low-action areas such as corridors. Secondly, the system would incorporate a fuzzy hash comparison using a Hamming distance threshold to perceptually hash the comparisons, allowing it to detect malicious replay attacks with modifications such as crops, rotations, or noise injection to evade exact matches. Thirdly, the backend component of the system would be expanded to incorporate multiple cameras to perform spatiotemporal correlations to detect anomalies in a network of cameras, which would be necessary to secure a larger-scale surveillance system.

REFERENCES

[1] D. Tran et al., "Learning Spatiotemporal Features with 3D Convolutional Networks," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2015, pp. 4489–4497.
[2] A. Hampapur et al., "Smart Video Surveillance: Exploring the Concept of Multiscale Spatiotemporal Tracking," IEEE Signal Process. Mag., vol. 22, no. 2, pp. 38–51, Mar. 2005.
[3] J. C. Nascimento and J. S. Marques, "Performance Evaluation of Object Detection Algorithms for Video Surveillance," IEEE Trans. Multimedia, vol. 8, no. 4, pp. 761–774, Aug. 2006.
[4] Y. Wang et al., "Replay Attack Detection Based on Parity Space Method for Cyber Physical Systems," IEEE Trans. Ind. Informat., vol. 19, no. 4, pp. 4567–4576, Apr. 2023.
[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Commun. ACM, vol. 60, no. 6, pp. 84–90, Jun. 2017.
[6] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks Against State Estimation in Electric Power Grids," ACM Trans. Inf. Syst. Secur., vol. 14, no. 1, pp. 13:1–13:33, May 2011.
[7] A. D. Wood et al., "Context Aware Wireless Sensor Networks for Assisted Living and Residential Monitoring," IEEE Netw., vol. 22, no. 4, pp. 26–33, Jul.–Aug. 2008.
[8] M. A. Ferrag et al., "Security for 4G and 5G Cellular Networks: A Survey of Existing Authentication and Privacy Preserving Schemes," J. Netw. Comput. Appl., vol. 101, pp. 55–82, Jan. 2018.
[9] A. K. Sahu, S. S. Manvi, and M. S. Kakkasageri, "A Survey on Security and Privacy Issues in Internet of Things," IEEE Internet Things J., vol. 4, no. 5, pp. 1250–1258, Oct. 2017.
[10] W. Sultani, C. Chen, and M. Shah, "Real-World Anomaly Detection in Surveillance Videos," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 6479–6488.
[11] M. Sabokrou et al., "Deep-Anomaly: Fully Convolutional Neural Network for Fast Anomaly Detection in Crowded Scenes," Comput. Vis. Image Underst., vol. 172, pp. 88–97, 2018.
[12] Y. Cong, J. Yuan, and J. Liu, "Sparse Reconstruction Cost for Abnormal Event Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2011, pp. 3449–3456.
[13] "UCF Crime Dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/odins0n/ucf-crime-dataset